

★ NOTE ★

Go through Textbook once, I
may have missed some topics!

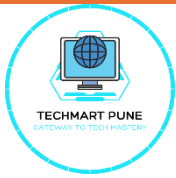
PEACE



itz-dk.28

Shot on OnePlus | **HASSELLBLAD**

By Dk | 27 August 2024 at 9:01 pm



Study material provided by: Vishwajeet Londhe

Join Community by clicking below links



Telegram Channel



https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)

@SPPU_TE_BE_COMP



WhatsApp Channel

(for all tech updates)



<https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b>



Insta Page

(for all engg & tech updates)



@SPPU_ENGINEERING_UPDATE

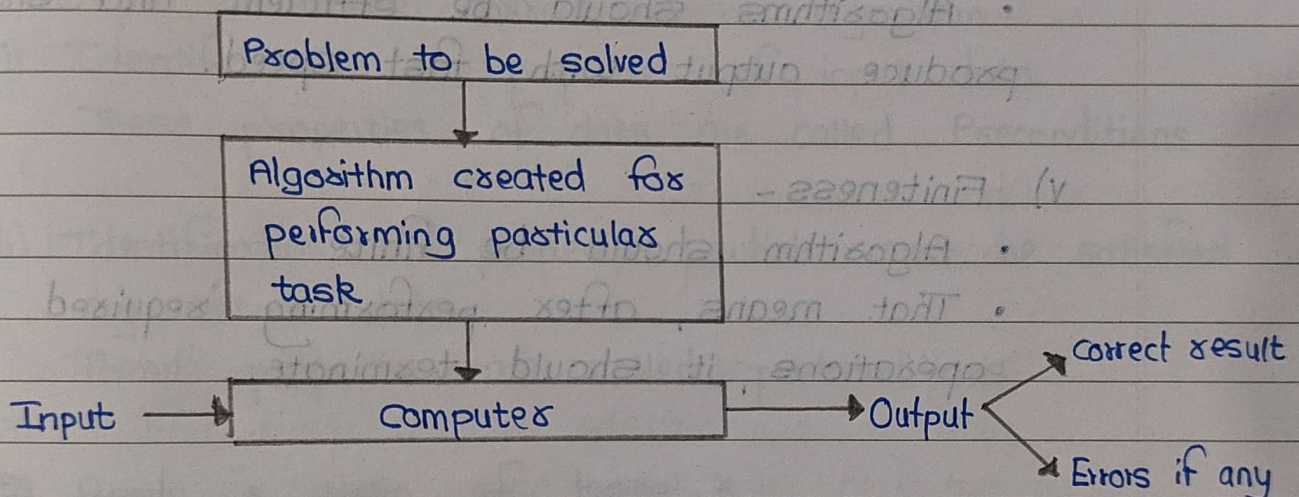
https://www.instagram.com/sppu_engineering_update

UNIT-I

Algorithms & Problem Solving* Algorithm -

→ • An algorithm is a finite set of unambiguous steps needed to be followed in certain order to accomplish a specific task.

• Diagram -



• Properties of Algorithm -

i) Non-ambiguity:

• Each step in an algorithm should be non-ambiguous

• That means, each instructions should be clear and precise.

ii) Range of input:

• The range of input should be specified.

• This is because, normally an algorithm is input driven & if range is not specified then algorithm can go in infinite state.

iii) Multiplicity -

- The same algorithm can be represented in multiple ways.
- That means, we can write sequence of instructions in simple English or we can write pseudo-code.

iv) Speed -

- Algorithms should be efficient and should produce output with fast speed.

v) Finiteness -

- Algorithm should be finite
- That means, after performing required operations, it should terminate.

★ Correctness of Algorithm -

→ • To ensure that algorithm is developed to correctly satisfy functional requirements, there is a need for correctness of algorithm.

- To ensure that solution for given problem is valid, we need to check correctness of algorithm.
- To check, if the given problem has finite or infinite solution, there is a need to check correctness of algorithm.

- The need for correctness of algorithm is for efficient execution of given task on computers

- To ensure that all the cases in problem statements are covered, it is necessary to check correctness of algorithm.

- Steps to check correctness of algorithm -

- i) Identification of properties of input data.

These properties of data are called Preconditions

- ii) Identification of properties which must be satisfied by output data.

These properties are called postconditions

iii) Apply a chain of logical &

- iii) Proving that starting from preconditions & executing each step specified in algorithm's one obtains the postconditions.

* Loop Invariant Property -

- • A loop invariant property is a property of an iterative algorithm.
- These algorithms use a loop control structure to define iterations in it.
- The loop invariant relation is true, before, after & during all iterations of iterative algorithm.
- After each iteration of loop, it gives an idea about current progress towards final output.
- To prove correctness of iterative algorithm, we need to demonstrate that a postcondition is satisfied upon termination of given loop.
- For the same, we need to show the following:
 - i) Initialization -
 - Loop invariant holds before first iteration of loop
 - ii) Maintenance -
 - If loop invariant holds at beginning, then it must also hold at termination of that iteration.
 - iii) Termination -
 - Loop always terminates after finite number of iterations.

iv) Correctness -

- Whenever loop invariant & termination condition of loop both hold, then postcondition must hold.

Q] Prove the correctness of finding summation of 'n' numbers using Loop Invariant Property.

→ • Invariant -

At start of each iteration of loop, variable 'sum' holds sum of first 'i-1' numbers

Proof of correctness -

i) Initialization:

- Before loop starts, loop counter 'i' is initialized to 1 and 'sum' is initialized to 0

- Hence invariant holds, sum of first 0 numbers is indeed 0.

ii) Maintenance -

- Assume that invariant holds true at beginning of i^{th} iteration.

i.e 'sum' holds sum of first 'i-1' numbers

- During i^{th} iteration, value of i is added to 'sum'

- After this addition, 'sum' now holds sum of first 'i' numbers

Algorithm -

def sum_num(n):

sum = 0

for i in range(1, n+1):

sum += i

return sum

- Hence, invariant holds, at beginning of $i+1^{th}$ iteration, 'sum' correctly holds sum of first 'i' numbers.

iii) Termination -

- Loop terminates after n^{th} iteration.
- At this point, loop invariant guarantees that 'sum' contains sum of first 'n' numbers.

iv) Correctness -

- Since, loop invariant holds throughout and loop terminates after summing up all numbers from 1 to n, algorithm is correct.

* Proof by mathematical Induction -

→ • This method is generally used to prove correctness of recursive algorithms.

- It is a very powerful technique of verifying an infinite number of facts in finite amount of space.

■ Procedure of PMI:

i) Basis Step -

- Prove directly that statement $A(n_0)$ is true for some base case n_0 where n_0 is integer.

ii) Induction Hypothesis -

- Assume that $A(n)$ holds for arbitrary integer $n > n_0$ & prove its implication that $A(n+1)$ holds for $\forall n \geq n_0$

iii) Inductive Step -

- Then by using principle of induction, statement $A(n)$ is true $\forall n \geq n_0$

Q] Prove the correctness of this algorithm using principle of mathematical induction or otherwise

- • We want to prove that $\text{sqr}(n)$ indeed computes n^2 for all non-negative integers n

I) Basis Step -

- We start by verifying base case, when $(n = 0)$

- For $(n = 0)$:

- According to algorithm, $\text{sqr}(0) = 0$
- square of 0 is $0^2 = 0$

- Thus base case holds true

Algorithm

```
int sqr(int n)
```

```
{
```

```
    if (n == 0) return 0;
```

```
    else return (2n +
```

```
        sqr(n-1));
```

```
}
```


II) Inductive Hypothesis -

- Assume that algorithm correctly computes square for some arbitrary +ve integer (k).
i.e. $\text{sqr}(k) = k^2 \rightarrow \textcircled{1}$

- We need to show that, algorithm will also correctly compute square for (k+1).
- For $n = k+1$, algorithm gives -

$$\text{sqr}(k+1) = 2 * (k+1) + \text{sqr}(k) - 1$$

Look at algorithm if you don't understand

Substituting $\textcircled{1}$,

$$\text{sqr}(k+1) = 2 * (k+1) + k^2 - 1$$

$$\text{sqr}(k+1) = 2k + 2 + k^2 - 1$$

$$\text{sqr}(k+1) = k^2 + 2k + 1$$

This can be factorized as,

$$\text{sqr}(k+1) = (k+1)^2$$

- Thus, $\text{sqr}(k+1) = (k+1)^2$, which means algorithm correctly computes square of (k+1).

III) Inductive Step -

- By principle of mathematical induction, algorithm correctly computes (n^2) for all non-negative integers (n).

★ Iterative - Algorithm & Its Design Issues -

- • Iterative algorithm is an algorithm which has at least one iterative component or loop.
- Even small amount of time spent on execution of loop will directly affect efficiency of overall algorithm.
- This situation can be worst, if nested loops is present in algorithm.
- Design Issues of Iterative Algorithm-
 - i) - Correct use of loops in the program
 - Iterative Algorithm uses looping control structure to implement iterations.

• For any loop, we specify:

- 1) Initial condition
- 2) Invariant Relation
- 3) Terminating Condition

• Eg, $i := 1$ // Initial condition
 $Sum := 0$
 while ($i \leq n$) // Terminating condition
 {
 $Sum := Sum + i$ // Invariant Relation
 $i := i + 1$;
 }

ii) Improving efficiency of algorithms -

- Considering limited computing resources, we should incorporate different techniques to design an efficient algorithm that consumes lesser time & memory.

- Some of the techniques are:

- 1] Removing redundant computations inside loop
- 2] Referencing array element
- 3] Late detection of terminating condition
- 4] Early detection of output condition

iii) Estimating and Specifying Execution Time -

- Time complexity is amount of time required by algorithm to complete

- Time complexity is given in terms of Frequency count, which denotes how many times particular statement is executed.

iv) Applying different algorithmic strategies -

- Problem solving strategy is general approach by which many problems can be solved algorithmically

- Some of them are, Divide & Conquer, Backtracking, Dynamic Programming, Greedy Technique, etc.

* Problem-Solving Strategies *

→ I) Brute Force method -

- It enumerates all possible solutions to a given problem without applying any heuristics

- It is a straightforward technique with naïve approach

II) Exhaustive Search -

- It generates all candidate solutions to a given combinatorial problem and identifies feasible solution

III) Divide and Conquer -

- Divide - Apply top-down approach to divide a large problem into smaller sub-problems.

- Conquer - Solve the sub-problems independently & recursively by invoking same algorithm

- Combine - Apply bottom-up approach to combine solutions of all sub-problems to get final solution

IV) Dynamic Programming -

- It is suitable for solving optimization problems with over-lapping sub-problems.

- The results of smaller, recurring instances are obtained to solve the problem.

* Algorithm as Technology *

→ • Algorithms are used as a tool to utilize reasonable amount of execution time & memory space.

• Different algorithms can be used in same problem. Each algorithm may take different amount of time

• Eg, For sorting, bubble sort algorithm requires more time than merge sort or quick sort.

• Execution of algorithm depends on 2 factors:

i) Selection of efficient algorithm

ii) Fast hardware upon which algorithm is to be executed.

• Hardware technology plays an important role in algorithm execution.

• Algorithms are core or fundamental factors for following technologies:

i) Advanced computer architectures

ii) Object oriented systems

iii) Applications that use GUI

iv) Integrated web technologies.

Analysis of Algorithms & Complexity Theory

* Space Complexity -

→ • Space complexity can be defined as amount of memory required by algorithm to run.

• Space requirement $S(p)$ can be given as,

$$S(p) = C + S_p,$$

where C , is fixed part & denotes space of inputs & outputs

S_p , is variable part whose space requirement depends on particular problem instance.

• The fixed part includes space for,

- Instructions

- Variables

- Array Size

- Constants

• Variable part includes space for, variables whose size is dependent upon particular problem instance being solved.

• Eg, Algorithm $\text{sum}(a, n)$

```
{
    s := 0.0;
```

```
    For i := 1 to n do
```

```
        s := s + a[i];
```

```
    return s;
}
```


In given code, we require space for,

$s := 0 \quad \leftarrow O(1)$

For $i := 1$ to $n \quad \leftarrow O(n)$

$s := s + a[i]; \quad \leftarrow O(n)$

return $s; \quad \leftarrow O(1)$

Hence, space complexity of given algorithm can be denoted in terms of big-oh notation. It is $O(n)$.

* Big-Oh Notation -

→ It is the method of representing upper bound of algorithm's running time.

- Using Big-Oh, we can give longest amount of time taken by algorithm to complete.

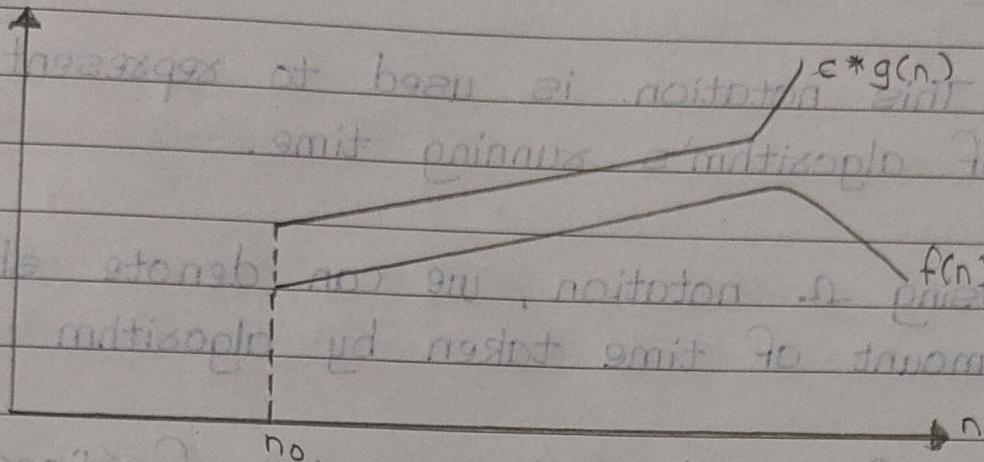
- Let $f(n)$ and $g(n)$ are two functions.

The function,

$f(n) = O(g(n))$ iff, there exists some positive constants c and n_0 such that,

$$f(n) \leq c * g(n), \quad \forall n \geq n_0$$

- Diagram -



- Eg, $f(n) = 2n + 2$, $g(n) = n^2$, then we have to find some constant 'c', so that $f(n) \leq c * g(n)$.

For $n = 1$,
 $f(n) = 2(1) + 2 = 4$
 $g(n) = 1^2 = 1$
 i.e. $f(n) > g(n)$

For $n = 2$
 $f(n) = 6$
 $g(n) = 4$
 i.e. $f(n) > g(n)$

For $n = 3$
 $f(n) = 8$
 $g(n) = 9$
 i.e. $f(n) < g(n)$

$\therefore$ We can conclude that for $n > 2$, we obtain
 $f(n) < g(n)$

* Omega Notation (Ω) -

→ • This notation is used to represent lower bound of algorithm's running time.

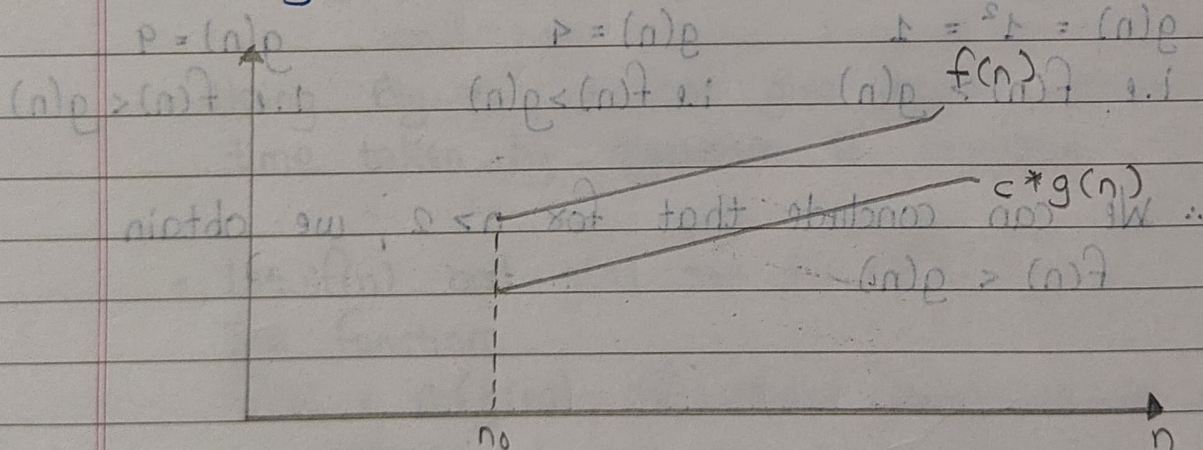
• Using Ω notation, we can denote shortest amount of time taken by algorithm

• Let $f(n)$ and $g(n)$ are two functions.

The function,

$f(n) = \Omega(g(n))$ iff, there exists some positive constants c and n_0 such that,
 $f(n) \geq c * g(n), \forall n \geq n_0$

• Diagram -



• Eg, $f(n) = 2n^2 + 5$, $g(n) = 7n$, then

for $n = 1$
 $f(n) = 2(1)^2 + 5 = 7$
 $g(n) = 7(1) = 7$
 $\therefore f(n) = g(n)$

for $n = 2$
 $f(n) = 9$
 $g(n) = 14$
 $\therefore f(n) < g(n)$

for $n = 3$
 $f(n) = 23$
 $g(n) = 21$
 $\therefore f(n) > g(n)$

Thus for $n > 3$ we get $f(n) > c * g(n)$.

* Theta Notation (Θ) -

→ • This method represents running time between upper bound and lower bound.

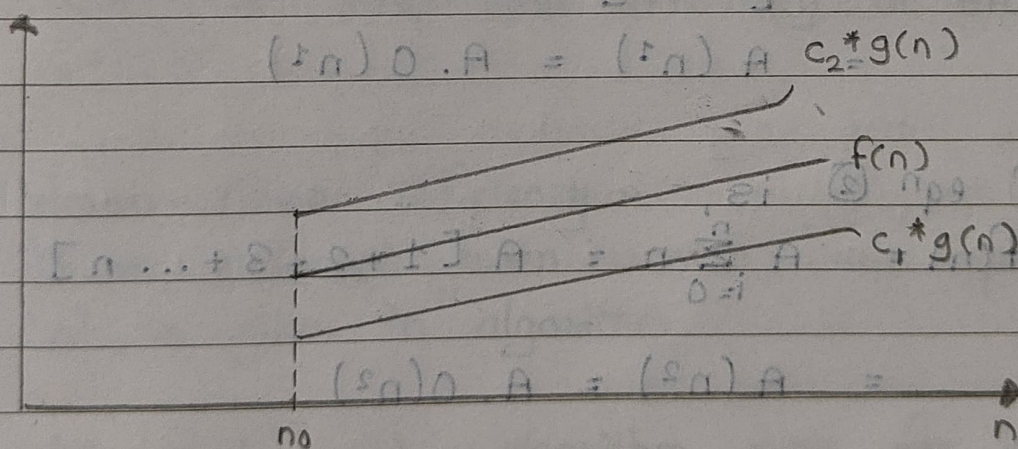
• Let $f(n)$ & $g(n)$ are two functions.

The function,

$f(n) = \Theta(g(n))$ iff, there exists two positive constants ' c_1 ' and ' c_2 ' such that,

$$c_2 * g(n) \leq f(n) \leq c_1 * g(n)$$

• Diagram -



• Eg, $f(n) = 2n + 8$, $g(n) = 5n$ where $n \geq 2$
 Similarly $f(n) = 2n + 8$, $g(n) = 7n$

i.e. $5n < (2n + 8) < 7n$ for $n \geq 2$

Hence $c_1 = 5$, $c_2 = 7$ & $n_0 = 2$

Date ____ / ____ / ____

Q] Prove that: $f(n) = a_m n^m + \dots + a_1 n + a_0$, then $f(n) = O(n^m)$.

→ Let,

$$f(n) = a_m n^m + a_{m-1} n^{m-1} + a_{m-2} n^{m-2} + \dots + a_0 \quad \text{--- ①}$$

If we treat a_i as constant then ① becomes,

$$f(n) = A \sum_{i=0}^m a_i n^i \quad \text{--- ②}$$

where $A = a_0 + a_1 + a_2 + \dots$

If eqn ② is,

$$\begin{aligned} f(n) &= A \left[\sum_{i=0}^n 1 \right] = A [1 + 1 + 1 + \dots + 1] \\ &= A(n^1) = A \cdot O(n^1) \end{aligned}$$

If eqn ② is,

$$\begin{aligned} f(n) &= A \sum_{i=0}^n n = A [1 + 2 + 3 + \dots + n] \\ &= A(n^2) = A \cdot O(n^2) \end{aligned}$$

If eqn ② is

$$f(n) = A \sum_{i=0}^n n^2 = A \cdot O(n^3)$$

Thus $f(n) = A \sum_{i=0}^m n^i = A \cdot O(n^m)$

Neglecting constant, we have,
 $f(n) = O(n^m)$

★ Best Case, Worst Case & Average Case -

→ i) Best Case Efficiency -

- It is achieved when the best-case input of size 'n' is given to algorithm.
- With such input, algorithm runs to its completion in least amount of time.

It gives the lower bound.

- Eg, In sequential search, if first element of given input of size 'n' is same as search key, then algorithm ends with successful search in just 1 comparison.
 $\therefore O(1)$ is best case efficiency of sequential search.

ii) Worst - Case Efficiency -

- It is achieved when worst-case input of size 'n' is given to algorithm.
- With such input, algorithm runs for longest, among all inputs.

It gives the upper bound.

- Eg, In sequential search, if last element of given input of size 'n' is equal to search key, then algorithm ends with successful search in 'n' comparison.
 $\therefore O(n)$ is worst-case efficiency of sequential search.

iii) Average-case efficiency -

- It is achieved when neither best-case nor the worst-case but any 'random' input of size 'n' is given to an algorithm.

- It is not calculated by averaging the worst-case and best-case efficiencies.

- Sometimes, co-incidentally the average case efficiency matches with mean of its worst-case & best-case efficiencies.

- Estimation of average-case efficiency is based on some probabilistic assumptions & is comparatively more complex than worst-case & best-case.

- Eg, for any random input of size 'n', sequential search performs 'n' comparisons. Average-case efficiency is $O(n)$.

★ P - Class Problem -

→ • It is the class of decision problems that can be solved in polynomial time by deterministic algorithms.

- A polynomial-time algorithm is an algorithm whose running time is polynomially dependent on input size of problem instance.
- Thus, polynomial-time algorithm for problem instance of size 'n' has its worst-case complexity $O(p(n))$, where $p(n)$ is polynomial of input size 'n'.
Eg, $O(n^2)$, $O(n^3)$, etc.

- P-class problems are decidable and tractable problems

• Eg of P-Class,

Linear Search: $O(n)$, Prim's Algorithm: $O(n^2)$,
Binary search: $O(\log n)$, etc.

★ NP - Class Problem -

→ • It is the class of decision problems that can be solved in polynomial time by non-deterministic algorithms.

- Non-deterministic polynomial-time algorithms produce possible solutions to given problems in non-deterministic way & verify correctness of solutions in polynomial time.

- NP-Class problems are decidable and tractable / intractable.

- All P-Class problems & NP-Complete problems are NP-Class problems, but its inverse is not true by assuming $P \neq NP$.

- Eg of NP-Class are, 0/1 Knapsack problem, TSP, etc.

★ NP-Hard Class -

- • A decision problem P_1 is NP-Hard if each NP-Class problem is polynomially reducible to P_1 , (i.e. for each problem $P_2 \in NP$, $P_2 \propto P_1$)

- All NP-Complete problems are NP-Hard problems, but all NP-Hard problems are not NP-Complete problems.

- NP-Hard problems are decidable or undecidable

- Eg, Decidable NP-Hard problems: TSP, vertex-cover decision problem

Undecidable NP-Hard Problems: Halting problem.

★ NP - Complete Class -

- • A decision problem P_1 is NP-complete if,
- P_1 is an NP-Class problem ($P_1 \in NP$).
 - Each NP-Class problem is polynomially reducible to P_1 (i.e. for each problem $P_2 \in NP$, $P_2 \propto P_1$).
- Thus P_1 is said to be complete if, $P_1 \in NP$ and $P_1 \in NP\text{-Hard}$.
- NP-Complete problems can have possible solutions only by applying non-deterministic algorithms & their solutions can be verified in polynomial time.
 - Eg of NP-Complete problem, Vertex-cover decision problem, TSP decision problem.

★ SAT and 3-SAT problem -

- • An expression in propositional calculus is considered to be 'satisfiable' if there is some set of truth assignments to variables in it that makes expression TRUE.
- Eg, $[\neg x_1 \wedge x_2]$ is satisfied if, $x_1 = \text{FALSE}$ and $x_2 = \text{TRUE}$

- The satisfiability (SAT) problem is to ascertain whether given Boolean expression is satisfiable
- Eg, $(\neg x, \neg x)$ is un-satisfiable for any values of x
- A 3-SAT problem is a problem which takes a Boolean formula 'S' with each clause having exactly 3 literals & checks if S is satisfied or not.

■ Proof that 3-SAT is NP-Complete:

- We replace each clause C that represents SAT problem to a function 'f' by family of ' D_c ' that represent satisfiability

- Eg, $C = a \vee b \vee c \vee d \vee e$

We can simulate this by,

$$D_c = (a \vee b \vee x) \wedge (\bar{x} \vee c \vee y) \wedge (\bar{y} \vee d \vee e)$$

where x and y are new variables

- We need to verify:

- IF C is false, D_c is FALSE; and
- IF C is TRUE, then one can make D_c TRUE

- If 'f' is satisfiable, then there is assignment where each clause C is TRUE. This can be extended to make D_C TRUE = ((n)0)0 at n = (n)1
- If 'f' is evaluated to FALSE, then some clauses, say 'c' must be FALSE & thus D_C evaluates to FALSE
- This conversion process can be done in polynomial time.
- As Thus, we've shown that SAT reduces to 3-SAT in polynomial time.
- As, SAT is NP-Complete problem, we must say that 3-SAT is also NP-Complete Problem.

Q] If $f(n) = O(g(n))$, then does it imply $g(n) = O(f(n))$.

→ • No, if $f(n) = O(g(n))$, it does not necessarily imply that $g(n) = O(f(n))$.

- $f(n) = O(g(n))$ means that $f(n)$ grows at most as fast as $g(n)$ asymptotically.
i.e. $f(n) \leq c * g(n)$

- $g(n) = O(f(n))$ would mean that $g(n)$ grows at most as fast as $f(n)$.
i.e. $g(n) \leq c' * f(n)$

Date ____ / ____ / ____

• Eg, consider $f(n) = n$ and $g(n) = n^2$.

$f(n) = n$ is $O(g(n)) = O(n^2)$ because,
 $n \leq c * n^2$, for $c = 1$, $n_0 = 1$

However, $g(n) = n^2$ is not $O(f(n)) = O(n)$,
because n^2 grows faster than any constant
multiple of n

$\therefore g(n) \neq O(f(n))$